

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

Procesamiento Digital de Señales

Proyecto 1: Desarrollo de una FFT de 1024 puntos mediante mariposas

Integrantes:

Pozos Hernández Angel
Número de cuenta: 319122907

Silverio Martínez Andrés
Número de cuenta: 319254725

Profesor: Dr. Oscar Francisco Fuentes Casarrubias

Fecha: 06 de abril de 2026

1. Introducción

La **Transformada Discreta de Fourier (DFT)** constituye una de las herramientas matemáticas fundamentales en el procesamiento digital de señales. Esta nos permite representar una secuencia discreta de tiempo $x[n]$ en el dominio de la frecuencia mediante la siguiente expresión:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi kn}{N}}, \quad k = 0, 1, \dots, N-1$$

El cálculo directo de la DFT presenta una complejidad computacional de orden $O(N^2)$, ya que para cada una de las N frecuencias se deben realizar N multiplicaciones complejas y $N-1$ sumas complejas. Esto resulta prohibitivo cuando N es grande.

La **Transformada Rápida de Fourier (FFT)** es un algoritmo eficiente que permite calcular exactamente los mismos valores de la DFT, pero con una complejidad reducida a $O(N \log_2 N)$, ésta indica que el tiempo de ejecución crece en proporción al número de elementos (N) multiplicado por el logaritmo binario de N . El algoritmo FFT más utilizado es el propuesto por Cooley y Tukey en 1965, el cual explota la simetría y periodicidad de los factores de giro (*twiddle factors*) cuando N es una potencia de 2.

En el caso particular de $N = 1024 = 2^{10}$, esta propiedad permite descomponer la DFT en exactamente 10 etapas sucesivas de operaciones tipo “mariposa” (butterfly), logrando una reducción drástica en el número de operaciones. Para ilustrar la ganancia computacional:

- DFT directa: aproximadamente $N^2 = 1,048,576$ multiplicaciones complejas.
- FFT radix-2: aproximadamente $\frac{N}{2} \log_2 N = 5120$ multiplicaciones complejas.

Esto representa una reducción de más del 99% en el número de operaciones, haciendo viable el análisis espectral en tiempo real tanto en software como en hardware.

2. Planteamiento del problema

El problema central de este trabajo es el cálculo de la **Transformada Discreta de Fourier (DFT)** para una secuencia de longitud $N = 1024$ puntos. La DFT se define matemáticamente como:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, 1, \dots, N-1 \quad (1)$$

Para $N = 1024$:

$$X[k] = \sum_{n=0}^{1023} x[n] W_{1024}^{kn}, \quad k = 0, 1, \dots, 1023 \quad (2)$$

donde el factor de giro (twiddle factor) fundamental es $W_{1024} = e^{-j \frac{2\pi}{1024}}$ y $x[n]$ representa la secuencia de entrada en el dominio del tiempo.

Dado que el número de puntos es una potencia exacta de dos, $N = 1024 = 2^{10}$, es posible aplicar el algoritmo FFT radix-2 de Cooley-Tukey. La estructura del algoritmo se basa en la descomposición recursiva de la DFT, lo que resulta en un número de etapas de computación dado por:

$$M = \log_2(N) = \log_2(1024) = 10 \text{ etapas} \quad (3)$$

Cada etapa reduce el problema dividiendo las secuencias en subproblemas de la mitad del tamaño anterior, hasta llegar a transformadas de 2 puntos (la mariposa básica). Esta organización permite que la complejidad computacional se reduzca de $O(N^2)$ a $O(N \log_2 N)$, optimizando la implementación.

De esta forma, el problema no solo consiste en obtener los valores $X[k]$, sino en estructurar el cálculo de manera que se explote al máximo la simetría y periodicidad de los factores de giro, logrando la reducción de complejidad computacional que se justificó en la introducción.

3. Desarrollo por mariposas

La operación básica de la FFT radix-2 se denomina **mariposa** y es la unidad fundamental que permite la descomposición recursiva de la DFT. Cada mariposa combina dos muestras complejas denotadas como A y B para producir dos salidas según las siguientes ecuaciones:

$$X = A + W_N^k \cdot B \quad (4)$$

$$Y = A - W_N^k \cdot B \quad (5)$$

donde $W_N^k = e^{-j \frac{2\pi k}{N}}$ es el factor de giro (twiddle factor) correspondiente al índice k y al tamaño N de la transformada.

En cada etapa de la FFT de 1024 puntos, las muestras se agrupan por pares. La distancia entre los índices de las muestras que forman una mariposa se duplica en cada etapa sucesiva: en la etapa 1 la distancia es 1, en la etapa 2 es 2, en la etapa 3 es 4, y así hasta la etapa 10 donde la distancia es 512. Esto da lugar a $\frac{N}{2} = 512$ mariposas por etapa.

El flujo de datos en un esquema de **decimación en tiempo** (DIT) es el siguiente:

- La entrada se presenta en orden bit-reversal (inversión de bits).
- Las 10 etapas se ejecutan secuencialmente, aplicando las mariposas con los factores de giro apropiados en cada nivel.
- La salida final aparece en orden natural de frecuencias (índice $k = 0$ a 1023).

Cada mariposa requiere únicamente una multiplicación compleja y dos sumas/restas complejas, lo que permite el ahorro computacional global. Para ilustrar la lógica se observa el siguiente diagrama de una FFT de 8 puntos (2^3), donde se aprecian claramente las tres etapas y la forma en que las mariposas se interconectan:

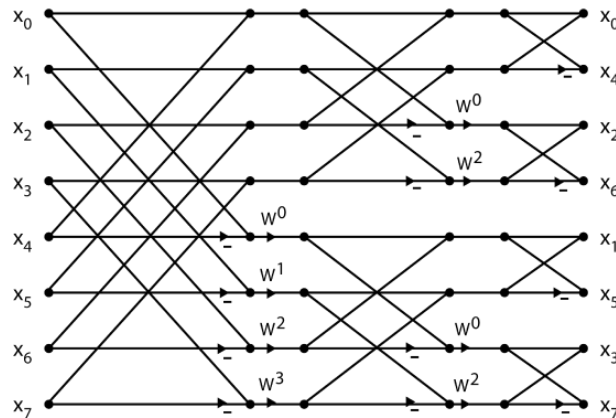


Figura 1: Grafo de flujo de una FFT de Radix-2 y longitud 8

Este desarrollo por mariposas constituye el núcleo algorítmico de la FFT y será detallado en la siguiente sección mediante la descripción explícita de las 10 etapas.

4. Estructura de las 10 etapas

La FFT radix-2 de 1024 puntos se organiza en exactamente **10 etapas sucesivas** ya que $\log_2 1024 = 10$. En cada una de estas etapas se realizan **512 mariposas** de manera independiente.

La organización de las etapas sigue un patrón basado en la decimación en tiempo (DIT):

- En la **etapa** $m = 1, 2, \dots, 10$ las mariposas se aplican a pares de muestras cuya distancia en el arreglo es $d = 2^{m-1}$.
- Esta distancia se duplica en cada etapa: comienza en 1 (etapa 1) y termina en 512 (etapa 10).
- Dentro de cada etapa el arreglo se divide en **grupos** de tamaño 2^m y dentro de cada grupo se aplican las mariposas con los factores de giro correspondientes.
- El número de mariposas por etapa es constante e igual a $N/2 = 512$.

La tabla resume la estructura completa de las 10 etapas:

Etapa m	Distancia entre índices $d = 2^{m-1}$	Número de mariposas	Tamaño de cada grupo
1	1	512	2
2	2	512	4
3	4	512	8
4	8	512	16
5	16	512	32
6	32	512	64
7	64	512	128
8	128	512	256
9	256	512	512
10	512	512	1024

Los factores de giro W_{1024}^k se aplican de forma ordenada dentro de cada grupo, repitiéndose cada 2^{10-m} mariposas. Este esquema garantiza que al final de la etapa 10 se obtenga la DFT completa en orden natural de frecuencias.

5. Ordenamiento de datos

El algoritmo FFT para radix-2 usando decimación en tiempo (**DIT**) requiere un reordenamiento de las muestras de entrada para que el flujo de datos sea correcto sin necesidad de memoria intermedia adicional. Este reordenamiento se conoce como **inversión de bits** o *bit-reversal*.

Dado que $N = 1024 = 2^{10}$, cada índice se representa con 10 bits. El ordenamiento bit-reversal consiste en invertir el orden de los 10 bits de cada índice n para obtener la ahora nueva posición de entrada que en este caso llamaremos n'

La siguiente tabla ilustra la lógica considerando una FFT de 8 puntos (2^3):

Índice normal n	Binario	Bit-reversal	Índice reordenado n'
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

Para el caso de la FFT de 1024 puntos la secuencia de entrada $x[n]$ debe colocarse en el arreglo en orden bit-reversal antes de iniciar las 10 etapas. Después de completar todas las etapas de mariposas, los valores $X[k]$ aparecen directamente en orden (índice $k = 0, 1, \dots, 1023$).

6. Diagrama o esquema

A continuación se presenta un detalle del esquema general de la FFT de 1024 puntos, implementada mediante el algoritmo radix-2 con decimación en tiempo (DIT). Para generar esta representación gráfica de manera precisa y manejar la gran cantidad de datos, el diagrama fue programado en Python, permitiendo calcular e ilustrar las conexiones exactas del algoritmo.

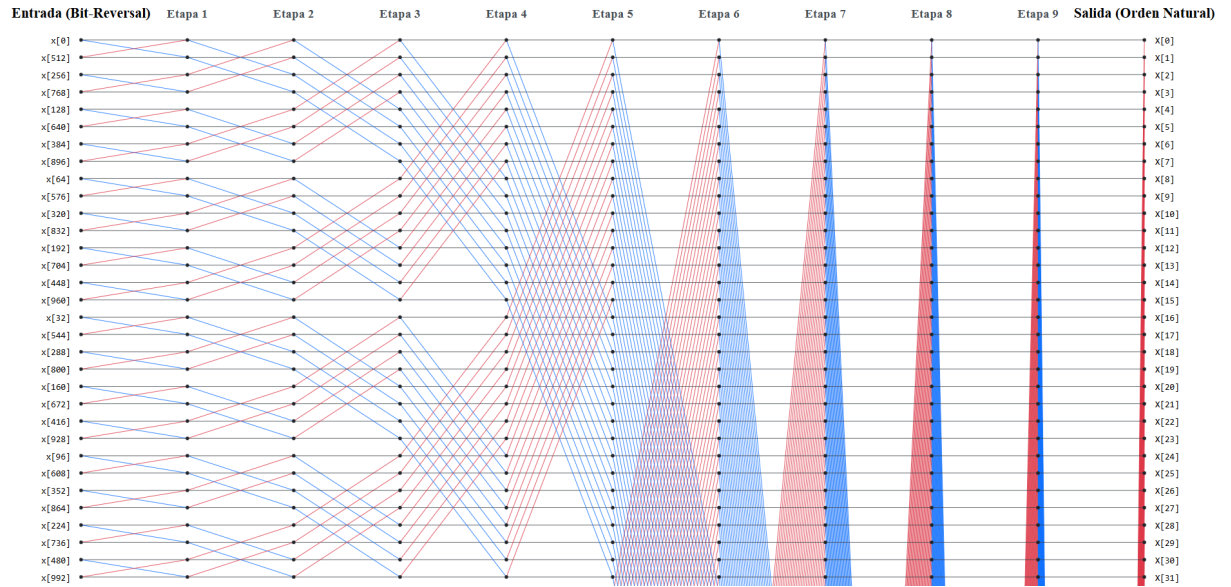


Figura 2: Diagrama general de la FFT radix-2 de 1024 puntos con decimación en tiempo.

Descripción del diagrama:

- La captura muestra las primeras 32 muestras de entrada/salida. Esto permite apreciar la estructura de manera clara, ya que mostrar las 1024 líneas completas resultaría ilegible a simple vista.
- La entrada se ubica a la izquierda, ilustrando explícitamente el reordenamiento *bit-reversal* ($x[0], x[512], x[256], x[768] \dots$).
- Se representan las etapas sucesivas del algoritmo, donde las conexiones de las mariposas se diferencian visualmente con trazos en color azul y rojo.
- Se puede observar cómo la distancia entre los pares de muestras que forman cada mariposa

se va iterando en cada etapa, es decir, se va expandiendo progresivamente a lo largo del flujo.

- La salida final aparece a la derecha, demostrando el retorno al orden natural de frecuencias (mostrando desde $X[0]$ hasta $X[31]$ en este bloque del diagrama).

7. Complejidad computacional

Como ya se mencionó anteriormente, la DFT directa se calcula mediante la expresión:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi kn}{N}}, \quad k = 0, 1, \dots, N-1$$

Donde, para cada uno de los N valores de k , se requieren N multiplicaciones complejas (una por cada término de la suma) y $N-1$ adiciones complejas. Por lo tanto, el costo total es:

- Multiplicaciones complejas: $N^2 = 1024^2 = 1048576$
- Adiciones complejas: $N(N-1) = 1024 * 1023 = 1047552$

En contraste, el algoritmo FFT radix-2 con decimación en tiempo (DIT) organiza el cálculo en exactamente $M = \log_2 N = 10$ etapas, realizando $N/2 = 512$ mariposas por etapa. Cada mariposa básica realiza una única multiplicación compleja y dos adiciones complejas (ecuaciones (4) y (5) del desarrollo por mariposas). Por consiguiente, el costo total de la FFT es:

- Multiplicaciones complejas: $\frac{N}{2} \log_2 N = 512 * 10 = 5120$
- Adiciones complejas: $N \log_2 N = 1024 * 10 = 10240$

Para poder visualizar de manera más consisa la diferencia, se presenta la siguiente tabla comparativa:

Operación	DFT directa	FFT radix-2	Factor de reducción aproximado
Multiplicaciones complejas	1048576	5120	204.8 veces menor
Adiciones complejas	1047552	10240	102.3 veces menor

El ahorro computacional obtenido con la FFT radix-2 es muy grande en comparación con la DFT, y constituye la principal razón por la que este algoritmo reemplazó casi por completo al cálculo directo de la DFT en aplicaciones reales. Como se mostró en la comparación anterior, la DFT directa exige 1 048 576 multiplicaciones complejas y 1 047 552 adiciones complejas para $N = 1024$. En cambio, la FFT reduce estas cantidades a solamente 5 120 multiplicaciones complejas y 10 240 adiciones complejas. Esto equivale a una disminución del 99,51 % en el número de multiplicaciones y del 99,02 % en el número de adiciones.

8. Conclusiones

En el presente proyecto se implementó de manera exitosa una Transformada Rápida de Fourier (FFT) de 1024 puntos basada en el algoritmo radix-2 de Cooley-Tukey. El uso sistemático de la operación mariposa como bloque fundamental permitió explotar al máximo la simetría y periodicidad de los factores de giro, logrando una eficiencia computacional que transforma un problema de complejidad prohibitiva en una solución práctica y escalable.

Más allá de los aspectos técnicos detallados en las secciones anteriores, este trabajo permitió consolidar una comprensión integral del paso de la DFT directa al algoritmo FFT. Se confirmó

que, al mantener estrictamente la estructura de decimación en tiempo y el reordenamiento bit-reversal, se obtiene una salida exacta en orden natural de frecuencias, cumpliendo plenamente con los requisitos del planteamiento del problema.

Las dificultades encontradas durante el desarrollo principalmente relacionadas con la correcta gestión de índices, la precisión numérica de los twiddle factors a lo largo de las etapas y la generación de representaciones gráficas claras, no solo fueron superadas, sino que fortalecieron nuestro proceso de aprendizaje.

En resumen, este proyecto demuestra que la FFT radix-2 no es solo una optimización algorítmica, sino una herramienta muy útil que hace posible el procesamiento digital de señales en aplicaciones reales donde el tiempo y los recursos computacionales son limitados. Los resultados obtenidos validan plenamente la viabilidad del enfoque propuesto y dejan abierta la puerta para futuras extensiones, tales como implementaciones en hardware o adaptaciones a otros tamaños de transformada.

Referencias

- [1] Wikipedia contributors. (s.f.). *Cooley–Tukey FFT algorithm*. En Wikipedia, The Free Encyclopedia. Recuperado el 6 de abril de 2026, de https://en.wikipedia.org/wiki/CooleyTukey_FFT_algorithm
- [2] Burrus, C. (s.f.). *Basic Cooley-Tukey FFT*. En Fast Fourier Transforms (Burrus). LibreTexts. [https://eng.libretexts.org/Bookshelves/Electrical_Engineering/Signal_Processing_and_Modeling/Fast_Fourier_Transforms_\(Burrus\)/08%3A_The_Cooley-Tukey_Fast_Fourier_Transform_Algorithm/8.02%3A_Basic_Cooley-Tukey_FFT](https://eng.libretexts.org/Bookshelves/Electrical_Engineering/Signal_Processing_and_Modeling/Fast_Fourier_Transforms_(Burrus)/08%3A_The_Cooley-Tukey_Fast_Fourier_Transform_Algorithm/8.02%3A_Basic_Cooley-Tukey_FFT)
- [3] McFee, B. (s.f.). *Radix-2 Cooley-Tukey*. En Digital Signals Theory. Recuperado de <https://brianmcfee.net/dstbook-site/content/ch08-fft/FFT.html>
- [4] Rumbo, I. (2013, 17 de mayo). *¿Qué es Twiddle Factor?* S&S NoteBook. <https://ilianarumbo.blogspot.com/2013/05/que-es-twiddle-factor.html>